Ders 5 Genişletilmiş Notu: Seminer Sunumları: Blok Tabanlı Görüntü Sıkıştırma, Elias ve Goldbach Kodları, Dil Modeliyle Sıkıştırma, MRLE ve CNN ile JPEG İyileştirme

Ders 5 Genişletilmiş Notu: Seminer Sunumları: Blok Tabanlı Görüntü Sıkıştırma, Elias ve Goldbach Kodları, Dil Modeliyle Sıkıştırma, MRLE ve CNN ile JPEG İyileştirme

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Seminer Dersinin Düzeni
- 2. Huffman Kullanan Neredeyse Kayıpsız Görüntü Sıkıştırma
- 3. Metin Sıkıştırma: RLE, Aritmetik Kodlama, Patlak Elias Kodu ve Goldbach Kodu
- 4. Dil Modelleme Yoluyla Metin Sıkıştırma
- 5. Veri Sıkıştırma Literatür Taraması
- 6. Mespotine RLE (MRLE) ve İyileştirilmiş MRLE (iMRLE)
- 7. CNN ile JPEG Sıkıştırma Kusurlarının Bastırılması
- Örnek Sorular
- Dersin Toparlaması

1. Seminer Dersinin Düzeni

Bu ders, bir önceki derste duyurulduğu gibi öğrenci seminerlerine ayrıldı. Her öğrenci veri sıkıştırmayla ilgili yayımlanmış bir makaleyi yaklaşık 15 dakikada anlattı. Hoca sunumlar sırasında araya girerek sorular sordu, sunumların sonunda da sınıfa soru olup olmadığını sordu. Hoca, sunumların makalenin künyesi ilk sayfada olacak biçimde paylaşılmasını istedi; böylece hangi makalenin anlatıldığı sonradan kolayca bulunabilir.

Toplam altı sunum yapıldı. Aşağıda her sunum ayrı bir bölüm olarak, anlatıldığı sırayla verilmiştir. Sunanlar “bir öğrenci” diye anılmıştır.

2. Huffman Kullanan Neredeyse Kayıpsız Görüntü Sıkıştırma

2.1 Problem ve amaç

İlk sunumu yapan öğrenci, görüntü sıkıştırma kalitesini artırmak için Huffman kodlamasını kullanan, neredeyse kayıpsız (near-lossless) bir tekniği anlatan bir makaleyi sundu. Problem şöyle tanımlandı: akıllı telefonlarla çekilen görüntülerin piksel sayısı ve boyutu arttıkça bu görüntüleri depolamak ve kısa sürede iletmek zorlaşır. Görüntü kalitesini koruyarak boyutu küçültme ihtiyacı sıkıştırmayı gerekli kılar.

Slayttaki giriş bölümüne göre önerilen yöntem (ekrandan, 00:10:00):

- Görüntünün farklı boyutlardaki bloklara bölünmesine dayanır: 2×2, 4×4 ve 8×8.
- Bloklardaki değerlere matematiksel işlemler uygulanır.

- Ardından Huffman kodlaması yapılır ve görüntü kalitesi mümkün olduğunca korunur.

Temel amaç, kayıplı sıkıştırmadan yararlanırken kayıpsız sıkıştırma türünü de kullanarak iyi bir kaliteyi korumak, yani iki türün avantajını birlikte kullanıp yüksek sıkıştırma oranı elde etmektir. Kayıpsız sıkıştırmada hiçbir veri kaybolmaz; kayıplıda biraz veri kaybolabilir ama daha çok sıkıştırma yapılır. Makalenin katkıları slaytta üç madde olarak verildi: neredeyse kayıpsız yeni bir dijital görüntü tekniği geliştirildi, sıkıştırma oranı ve kalite iyileştirildi, performans mevcut tekniklerle karşılaştırılarak doğrulandı.

Soru: Bu makaledeki resimler renk tablolu, yani 256 renkle sınırlı mı, yoksa 24 bitlik, üç ayrı renk paletli mi? **Cevap:** Yöntem hem gri tonlu görüntülerde hem de renkli görüntülerde uygulanmıştır.

2.2 Algoritma seçimi ve kayıpsız yöntemler

Makaleye göre bir sıkıştırma algoritması seçilirken önce görüntü tipine en uygun algoritma seçilmelidir. Veri sağlıkla ilgiliyse ya da hayati bilgi taşıyorsa kayıplı sıkıştırma kullanmak doğru değildir. Görüntü sıkıştırmanın avantajları olarak daha az maliyet, daha kısa iletim süresi, daha az güç ve daha az bit gereksinimi sayıldı.

Kayıpsız yöntemler tıbbi görüntüler gibi veri kaybının kabul edilemeyeceği durumlar için uygundur; sunumda örnek kayıpsız formatlar olarak BMP, GIF ve PNG sayıldı. Makalede kayıpsız algoritma olarak Huffman, RLE ve LZW anılmıştır. Huffman'da sık kullanılan karakterlere kısa, az kullanılanlara uzun kodlar verilir. LZW'de algoritma metin içinde harf harf ilerler, mümkün olan en uzun diziyi sözlüğe ekler ve bu sırada diziyi sözlükteki karşılığıyla değiştirir. Kayıplı yöntemler ise verinin birebir korunmasının önemli olmadığı ses sinyalleri ve görüntüler gibi veriler için önerilir.

2.3 Adımlar ve sayısal örnek

Önerilen tekniğin adımları şunlardır:

- Görüntü okunur ve bloklara bölünür (örnekte 8×8).
- Her bloğun en küçük değeri bulunur.
- Bu en küçük değer bloktaki bütün değerlerden çıkarılır.
- Tek olan değerlerden 1 çıkarılır.
- Bütün değerler 2'ye bölünür.
- Sonuca Huffman kodlaması uygulanır.

Slayttaki örnek (ekrandan, 00:15:15) 8×8'lik bir A bloğuyla başlar:

```
A (orijinal görüntü verileri)
188 231 188 230 188 230 188 230
230 188 231 194 231 188 231 188
195 230 188 230 188 230 188 230
230 188 231 188 189 230 188 189
189 188 230 188 230 194 188 230
230 188 189 230 188 230 189 188
230 188 230 188 188 230 188 230
188 231 188 230 189 188 230 188
```

Adım 1 (B tablosu). A'daki en küçük değer 188'dir. Her değerden 188 çıkarılır: $188-188=0$, $231-188=43$, $230-188=42$, $194-188=6$, $195-188=7$, $189-188=1$.

```
B (en küçük değer çıkarıldıktan sonra)
 0 43  0 42  0 42  0 42
42  0 43  6 43  0 43  0
 7 42  0 42  0 42  0 42
42  0 43  0  1 42  0  1
 1  0 42  0 42  6  0 42
42  0  1 42  0 42  1  0
42  0 42  0  0 42  0 42
 0 43  0 42  1  0 42  0
```

Adım 2 (C tablosu). Tek değerlerden 1 çıkarılır: 43 → 42, 7 → 6, 1 → 0. Böylece bütün değerler çift olur.

Adım 3 (D tablosu). C'deki bütün değerler 2'ye bölünür: 42 → 21, 6 → 3, 0 → 0.

```
D (2'ye bölündükten sonra)
 0 21  0 21  0 21  0 21
21  0 21  3 21  0 21  0
 3 21  0 21  0 21  0 21
21  0 21  0  0 21  0  0
 0  0 21  0 21  3  0 21
21  0  0 21  0 21  0  0
21  0 21  0  0 21  0 21
 0 21  0 21  0  0 21  0
```

Adım 4 (E tablosu). D'ye Huffman uygulanır. D'de yalnız üç farklı değer kalmıştır: 0, 21 ve 3. Sunumda anlatıldığı gibi en sık değer olan 0'a 0, 21'e 1, 3'e 2 simgesi verilmiştir; E tablosu bu simgelerle gösterilir (ekrandan, 00:15:15). Python ile sayıldığında D'de 33 tane 0, 28 tane 21 ve 3 tane 3 vardır (ara adım eklendi).

Soru: Makale tek sayılardan 1 çıkarmanın avantajını açıklıyor mu? Orijinal değerleri renk değerleri olarak kabul edersek, renk tekse ondan 1 çıkarmanın nasıl bir avantajı var? **Cevap:** Makale bunu açıklamıyor. Sunan öğrenciye göre bunun nedeni, bir sonraki adımda değerlerin 2'ye tam bölünebilmesidir. Hoca, kendi derste ağaç adreslerini yerleştirirken çift ve tek adreslerle benzer bir şey yaptıklarını hatırlattı.

> **Ek bilgi (derste anlatılmadı):** Yöntemin "neredeyse kayıpsız" olmasının nedeni büyük olasılıkla bu tek değerden 1 çıkarma adımıdır. Çözücü değeri $2D + \min$ diye geri kurar; orijinal fark tekse geri kurulan değer orijinalden 1 eksik çıkar. Böylece her pikseldeki hata en çok 1 gri seviyesi olur, buna karşılık alfabe yarıya iner ve Huffman daha kısa kodlar üretir.

2.4 Ölçütler ve sonuçlar

Karşılaştırmada iki standart ölçüt kullanılmıştır: ortalama kare hatası (MSE) ve tepe sinyal-gürültü oranı (PSNR). Huffman kayıpsız bir teknik olduğu için MSE değeri 0'dır. PSNR, MSE'ye bölmeyle hesaplandığından Huffman'ın PSNR'si sonsuz çıkar. Sunan öğrenciye göre renkli görüntülerde en düşük ortalama MSE 2×2 bloktadır (0,32; diğerleri 0,43 ve 0,45 civarı); gri tonlu görüntülerde de 2×2 blok en düşük MSE'yi (0,33) verir.

Sıkıştırma oranı (CR) slayttaki tanıma göre (ekrandan, 00:20:05) sıkıştırılmış çıktı görüntüsünün boyutunun girdi görüntüsünün boyutuna bölünmesiyle bulunur. CR sıfıra ne kadar yakınsa sıkıştırma o kadar iyidir. Renkli görüntüler için Tablo 5'teki ortalamalar:

Yöntem	Ortalama CR
Önerilen 2×2	0,94
Önerilen 4×4	0,54
Önerilen 8×8	0,48
Huffman	0,74

Slayttaki sonuç cümlesine göre 8×8 blok bütün blok boyutları arasında en iyi sıkıştırma oranını vermiş ve Huffman'a göre %26 daha fazla tasarruf sağlamıştır. Gri tonlu görüntülerde ise sunan öğrenciye göre en iyi oran 4×4 bloktadır ve yine Huffman'dan daha iyidir. Makalede ayrıca tabloların Friedman sıralama testiyle sıralandığı belirtildi; öğrenci bu testi tam anlayamadığını söyledi.

Soru (öğrencinin hocaya sorusu): Sıralamada bazen 4×4, bazen 8×8, bazen 2×2 daha iyi çıkıyor; bunun nedeni ne? **Cevap:** Hocaya göre görüntülerin hepsi aynı sayıda renk barındırmaz; gri tonlu görüntüde seviye sayısı sınırlıdır, renkli görüntüde renk sayısı farklıdır. Sonuçlar bununla ilişkilidir.

> **Ders sonrası not:** Sunumun sonunda 2×2 bloğun en iyi olduğu söylendi (00:22:25). Slayttaki Tablo 5'e göre (ekrandan, 00:20:05) en iyi sıkıştırma oranı 8×8 bloktadır (0,48); 2×2'nin ortalama oranı (0,94) Huffman'ınkinden (0,74) bile kötüdür. 2×2'nin üstünlüğü yalnız MSE, yani kalite bakımındandır.
>
> **Ders sonrası not:** Slayttaki "%26 daha fazla tasarruf" ifadesi $0{,}74 - 0{,}48 = 0{,}26$ farkından gelir; bu, CR'deki mutlak farktır. Huffman'ın çıktısına göre oransal küçülme $0{,}26/0{,}74 \approx \%35$'tir (python ile kontrol edildi).

Makale, bu yöntemin Huffman'a göre daha iyi olduğu ve gelecekte farklı matematiksel blok işlemleriyle daha iyi sıkıştırma yapılabileceği sonucuyla bitmektedir.

3. Metin Sıkıştırma: RLE, Aritmetik Kodlama, Patlak Elias Kodu ve Goldbach Kodu

3.1 Makale ve deney düzeni

İkinci sunum, dört kayıpsız metin sıkıştırma yöntemini karşılaştıran 2017 tarihli bir makaleyi ele aldı. Başlığı altyazıda bozuk geçmektedir; büyük olasılıkla "Compression of Text Data Using Run-Length Encoding, Arithmetic Encoding, Punctured Elias Code and Goldbach Code" biçimindedir.

Makale sıkıştırmayı kayıplı ve kayıpsız diye ikiye ayırır; amacı dört kayıpsız tekniği metin üzerinde karşılaştırmaktır. Test verisi 5, 10 ve 15 byte'lık metinlerdir.

> **Hocanın vurgusu:** Metinler çok küçük. 5 byte'lık bir şey sıkıştırılınca başlık (header) verinin kendisinden büyük olur.

3.2 RLE

RLE'de arka arkaya gelen semboller "tekrar adedi" ve sembol olarak kodlanır. Birçok varyasyonu vardır. Arka arkaya gelen sembollerin az olduğu verilerde başarılı değildir; dikkat edilmezse boyutu büyütür (ekrandan, 00:28:50).

Sunumdaki ilk örneklerde 9 tane `a` "a9" diye iki karaktere, 4 tane `e` "e4" diye kodlanır. Alfabede yalnız 0 ve 1 varsa sembol hiç yazılmaz, yalnız tekrar sayıları yazılır: 5 tane 0, 4 tane 1, 5 tane 0 dizisi "5 4 5" olur. Öğrencinin hazırladığı etkileşimli bir demoda tekrar sayısı 3'ü geçtikten sonra sıkıştırmaya başlanıyordu.

Kaçış kodlu örnek (ekrandan, 00:28:50):

```
Girdi : 8 8 8 8 8 96 12 45 45 45 45        (11 değer)
Çıktı : 255 5 8 96 12 255 4 45              (8 değer)
        255 = kaçış (escape) kodu, ardından adet, ardından değer
```

Beş tane 8 "255 5 8" diye kodlanır. 96 ve 12 tek oldukları için olduğu gibi geçirilir. Dört tane 45 "255 4 45" olur. Böylece 11 değerlik veri 8 değere iner.

3.3 Aritmetik kodlama

Aritmetik kodlama diğer yöntemlerden çok farklıdır: bütün mesaj 0 ile 1 arasında tek bir sayıyla kodlanır. Önce [0, 1) aralığı sembollerin frekansları oranında bölünür. İlk sembol hangisiyse bir sonraki adımın aralığı o sembolün aralığı olur; bu aralık yine frekanslar oranında bölünür ve işlem böyle sürer.

Slayttaki örnekte dört sembol vardır (ekrandan, 00:29:50): a = 0,2, b = 0,3, c = 0,1, d = 0,4. Kümülatif aralıklar a: [0; 0,2), b: [0,2; 0,5), c: [0,5; 0,6), d: [0,6; 1,0). Kodlanan dizi `b d a b`'dir:

- `b`: aralık [0,2; 0,5) olur. Genişlik 0,3.
- `d`: $0{,}2 + 0{,}3 \cdot 0{,}6 = 0{,}38$ ile $0{,}2 + 0{,}3 \cdot 1{,}0 = 0{,}5$; aralık [0,38; 0,5). Genişlik 0,12.
- `a`: $0{,}38$ ile $0{,}38 + 0{,}12 \cdot 0{,}2 = 0{,}404$; aralık [0,38; 0,404). Genişlik 0,024.
- `b`: $0{,}38 + 0{,}024 \cdot 0{,}2 = 0{,}3848$ ile $0{,}38 + 0{,}024 \cdot 0{,}5 = 0{,}392$; aralık [0,3848; 0,392).

Son aralıktaki herhangi bir sayı `b d a b` dizisini temsil eder (ara değerler slayttan, python ile doğrulandı). Öğrencinin demosunda ilk sembol `d` seçilseydi aralığın [0,6; 1) olacağı gösterildi.

Ders sonrası not: Sunumda d'nin frekansının %40, b'nin %30, c'nin %10 olduğu söylendi (00:29:46); a'nınki söylenmedi. Slayta göre a'nın olasılığı 0,2'dir.

3.4 Patlak (punctured) kodlar ve patlak Elias kodu

"Punctured" kavramı hata düzeltme kodlarından gelir. Bazı kodlamalarda veriye hata düzeltme bitleri eklenir; sonra bu bitlerin hepsine gerek olmadığına karar verilip bir kısmı atılır. Bu "delme" işlemine punctured denir; öğrenci Türkçe karşılık olarak "patlak kod" ifadesini kullandı.

Makalenin incelediği üçüncü yöntem patlak Elias kodudur (P1 varyantı). Slayttaki örnek (ekrandan, 00:32:45):

Adım	Değer
Sayımız	13
İkilik karşılığını al	1101
Bit sırasını ters çevir	1011
Her 1 için bir 1, sonuna 0 koyarak önek oluştur	1110
İki değeri birleştir	1110 1011
Kod	11101011

1011'de üç tane 1 olduğu için önek 111 olur, sonuna 0 eklenince 1110 elde edilir (python ile doğrulandı).

3.5 Elias gamma kodu

Goldbach kodlarında kullanıldığı için önce Elias gamma kodu anlatıldı. Bu kod, önek (prefix) kodların en basiti gibi düşünülebilir. Sayının ikilik karşılığının önüne (bit sayısı − 1) tane 0 konur (ekrandan, 00:33:40):

```
13 -> 1101 (4 bit) -> önüne 3 tane 0 -> 0001101
```

Bu kod çok kullanışlıdır: baştaki 0'ların sayısına bakarak sayının hangi aralıkta olduğu hemen anlaşılır. Üç tane 0 varsa sayı 4 bitliktir, yani 8 ile 15 arasındadır.

3.6 Goldbach kodları

Goldbach kodları bir matematik özelliğine dayanır: 2'den büyük her çift sayı iki asal sayının toplamı olarak yazılabilir. Öğrenci bu fikrin tarihini anlattı: 18. yüzyılda Christian Goldbach bunu bir mektupla Euler'e sormuş, Euler de bunun doğru olduğunu düşündüğünü ama kanıtlayamadığını söylemiştir. Bu ifade bugün de kanıtlanamamıştır; bu yüzden teorem değil konjektürdür. Kodları bir bilgisayar bilimcisi (slayta göre Peter Fenwick) geliştirmiştir (ekrandan, 00:36:30).

G0 kodu G0'da sıralı asal sayılar tablosu kullanılır: 1 → 3, 2 → 5, 3 → 7, 4 → 11, 5 → 13, 6 → 17, 7 → 19, 8 → 23 (ekrandan, 00:36:30). Kodlanacak sayı n ise:

- $2(n+3)$ hesaplanır.
- Bu sayı iki asalın toplamına ayrılır.
- Asal tablosundaki sırada bu iki asala denk gelen bitlere 1, diğerlerine 0 yazılır. Bu yüzden kodlarda hep iki tane 1 bulunur.

Örnek: $n = 3$ için $2(3+3) = 12 = 5 + 7$. Tablo sırası [3][5][7] için bitler 0, 1, 1 olur; kod **011**'dir.

Örnek: $n = 6$ için $2(6+3) = 18 = 7 + 11$. [3][5][7][11] için bitler 0, 0, 1, 1; kod **0011**'dir.

Slayttaki 1-10 arası sayıların G0 kodları (ekrandan, 00:37:20):

n	2(n+3)	Asal toplayanlar	Kod
1	8	3+5	11
2	10	3+7	101
3	12	5+7	011
4	14	3+11	1001
5	16	5+11	0101
6	18	7+11	0011
7	20	7+13	00101
8	22	5+17	0100001
9	24	11+13	00011
10	26	7+19	0010001

Ders sonrası not: Tablodaki $n = 8$ satırında ($22 = 5 + 17$) kod 0100001 yazıyordu. Sütun sırasına ([3][5][7][11][13][17]) göre bu kod 010001 olmalıdır; slayttaki fazladan 0 büyük olasılıkla yazım hatasıdır.

G1 kodu G1'in fikri şudur: sayı büyüdükçe toplamı veren iki asal sayı (i. ve j. asal) birlikte büyür ve genellikle birbirine yakın kalır. Bu yüzden ikinci indeks yerine iki indeksin farkı kodlanır: (i, j) çifti yerine $(i, j - i + 1)$ çifti kullanılır ve her biri Elias gamma koduyla yazılır. i normal biçimde büyür, ama $j - i + 1$ hemen hemen değişmez ve birkaç bitte kalır.

G1'in asal tablosu 1'i de içerir: 1 → 1, 2 → 3, 3 → 5, 4 → 7, 5 → 11, 6 → 13, 7 → 17, 8 → 19 (ekrandan, 00:39:10).

Örnek: $N = 8 = 3+5$. 3'ün indeksi 2, 5'in indeksi 3'tür; $(i, j) = (2, 3)$. Çift $(2, 3-2+1) = (2, 2)$ olur. $\gamma(2) = 010$ olduğundan kod **010 010**'dur.

Örnek: $N = 16 = 5 + 11$. İndeksler (3, 5), çift (3, 3), kod **011 011**'dir.

Slayttaki G1 tablosu (ekrandan, 00:39:10):

N	Değerler	İndeksler (i, j)	Çift (i, j−i+1)	Kod
2	1+1	1,1	1,1	1 1
4	1+3	1,2	1,2	1 010
6	3+3	2,2	2,1	010 1
8	3+5	2,3	2,2	010 010
10	3+7	2,4	2,3	010 011
12	5+7	3,4	3,2	011 010
14	7+7	4,4	4,1	00100 1
16	5+11	3,5	3,3	011 011
18	7+11	4,5	4,2	00100 010
20	7+13	4,6	4,3	00100 011

Ders sonrası not: Sunumda asalların birbirine yakın kaldığını göstermek için 1000 = 497 + 503 örneği verildi (00:38:00). 497 = 7 × 71 asal değildir; örnek büyük olasılıkla 1000 = 491 + 509 gibi iki asaldan oluşan bir çiftti (python ile kontrol edildi).

G2 kodu G1 yalnız çift sayılar için düşünülmüştür, bu yüzden genel bir kod değildir. G2 bütün sayılar için bir kod verir ve sayının özelliğine göre dört farklı yol izler:

- 1 ve 2 özel sayılardır; kodları sabittir, çünkü bu kadar küçük sayılar iki asala bölünemez.
- Çift sayılar G1'deki gibi Elias gamma kodlarıyla kodlanır; yalnız indeks çiftinde küçük bir değişiklik yapılır.
- Asal tek sayılarda sayının Elias gamma kodu bulunur ve sonuna 1 eklenir.
- Asal olmayan tek sayılarda sayının bir eksiğinin G2 kodu alınır ve başına 1 eklenir. Örneğin 15 asal olmayan tek sayıdır; 14'ün G2 kodu bulunur ve başına 1 konur.

Ders sonrası not: Sunumda 11'in Elias gamma kodunun "00101" olduğu söylendi (00:41:27). 11'in ikilik karşılığı 1011 (4 bit) olduğundan gamma kodu 0001011'dir; 00101, 5'in gamma kodudur (python ile kontrol edildi).

3.7 Sonuçlar ve sıkıştırma oranı tartışması

Makalenin sonuçlarına göre Goldbach kodu başarılı, RLE ise en başarısız sonucu vermiştir; RLE ile metin büyümüştür. 5 byte'lık (40 bit) test için slayttaki tablo (ekrandan, 00:43:00):

Yöntem	Girdi	Çıktı	Sıkıştırma oranı
RLE	40 bit	80 bit	%200
Aritmetik kodlama	40 bit	32 bit	%80
Patlak Elias kodu	40 bit	64 bit	%160
Goldbach kodu	40 bit	14 bit	%35

Soru: Bu %200, %160 sıkıştırma oranları ne? Niye %100'ün üstünde? **Cevap:** Sıkıştırma oranı çıktı/girdi formülüyle hesaplanmıştır. RLE ile 40 bit 80 bite çıktığı için oran %200'dür. Makale, sıkıştırma yerine büyüme olan yöntemlerde (RLE ve patlak Elias) oranın başına eksi koymuştur; grafikte bu yüzden -200 ve -160 görünür.

> **Hocanın vurgusu:** Formül zaten çıktı bölü girdidir. 40 bitten 80 bite çıkılıyorsa ortada sıkıştırma yoktur.

Hoca, makalenin işaret seçimine fazla girilmemesini, çünkü bunun nedeninin bulunamayacağını söyledi. Sunumun kazanımı olarak iki yeni önek kod öğrenildiğini belirtti: patlak Elias kodu ve Goldbach kodu.

4. Dil Modelleme Yoluyla Metin Sıkıştırma

4.1 Terimler

Üçüncü sunum, 2016'da Stanford NLP (doğal dil işleme) grubundan çıkan bir çalışmayı anlattı. Önce terimler tanımlandı:

- **Dil modeli:** kelimeler ya da kelime dizileri üzerindeki olasılık dağılımı.
- **n-gram:** n tane karakter ya da kelimeden oluşan dizi.
- **Korpus:** metinler koleksiyonu.

4.2 Fikir

Dil modeliyle metin sıkıştırma olasılıklara dayanır. Bir insan "he is" kalıbından sonra "going" gelme olasılığının "be" gelme olasılığından yüksek olduğunu kafasında otomatik olarak bilir. Çalışmanın hedefi, girdi dizisinin yapısı hakkındaki bu ön bilgiyi sıkıştırmaya dahil etmektir. Model düşük entropiye sahipse daha iyi sıkıştırma elde edilir.

4.3 Perplexity ve entropi

Slayttaki tanıma göre (ekrandan, 00:50:00) perplexity (çapraşıklık), olasılık modelinin bir örneği ne kadar iyi tahmin ettiğini gösterir:

$$\text{Perplexity} = 2^{H(X)}$$

İyi, yani düşük perplexity veren bir dil modeli, iyi sıkıştırmaya izin veren düşük entropili bir dağılım demektir. Entropi ortalama bir kelimenin kaç bitle ifade edilebileceğidir. Sunan öğrenci perplexity'yi "şaşkınlık" diye açıkladı: model bir kelimeyi ilk kez gördüğünde şaşırır; aynı kelimeyi tekrar tekrar gördükçe kelimenin olasılığı yükselir ve şaşkınlık azalır. Dil modeli ne kadar büyük bir korpusla eğitilirse o kadar iyidir.

4.4 Karakter kodlama ve kelime kodlama

Çalışmada asıl yöntem kelime kodlamadır; karakter kodlama bir geri çekilme (back-off) yolu olarak kullanılır. Sıkıştırılacak kelime korpusta yoksa, yani ilk kez görülüyorsa, karakter kodlamaya dönülür. Karakter kodlamada her dosya için ayrı bir Huffman ağacı kurulmaz; karakter

n-gramları için sabit bir Huffman kodu kullanılır. Bu sabit kod, dosyadaki karakter dağılımı eğitim korpusunun karakter dağılımına uyuyorsa optimaldir; uymuyorsa her dosya için ayrı Huffman kodlaması gerekir.

Kelime kodlamada eğitilmiş dil modelinden unigram, bigram ve trigram kelimeler için Huffman ağaçları kurulur. Unigram ağacında bir kelimenin olasılığı doğrudan kullanılır; bigram ya da trigram ağacında olasılık önceki bir ya da iki kelimeye bağlı olarak hesaplanır. Örneğin "to have" kalıbından sonra "been", "seen", "done" gelme olasılıkları vardır; olasılığı yüksek olan kelimeler ağacın üst kısmına yerleşir ve daha az bitle kodlanır. Ağaç kurulumu bilinen Huffman yöntemidir: en düşük olasılıklı iki eleman tek bir düğümde birleştirilir.

4.5 Sıkıştırma ve açma

Sıkıştırma başlangıçta iki token ile başlar. Her kelime için önceki iki kelimeye bağlı trigram Huffman ağacına bakılır:

```
kelime trigram ağacında var mı?
  evet -> Huffman kodunu yaz
  hayır -> unknown token yaz, unigram modeline dön
           unigram ağacında var mı?
             evet -> Huffman kodunu yaz
             hayır -> karakter kodlamaya dön
```

Bu geri dönüşler sıkıştırma performansını düşürür; algoritmanın performansı tamamen dil modelinin perplexity'sine, yani ne kadar iyi olduğuna bağlıdır. Huffman kullanıldığı için sıkıştırma kayıpsızdır. Açma işlemi bu adımların tersidir. Daha hızlı çalışmak için dengeli ağaçlar gibi veri yapılarının (STL map) kullanılabileceği belirtilmiştir; çünkü bir kelimenin ağaçta olup olmadığı sürekli sorgulanır.

4.6 Uzantılar

Makalede iki uzantı önerilmiştir, ama bunlar denenmemiş, teorik kalmıştır:

- **Belgelerin kategorize edilmesi:** spor, siyaset, ekonomi gibi kategorilere özel modeller kurulursa sözlük küçülür, konuya özgü kelimeler daha olası hale gelir, bilinmeyen token ile karşılaşma azalır; sonuç daha düşük entropi ve daha iyi sıkıştırmadır.
- **Kayan bağlam penceresi:** örneğin 1000 kelimelik bir pencere üzerinde çalışılırsa sorgular daha hızlı olur ve konu belge boyunca değiştikçe daha özel bir sözlük kullanılmış olur.

4.7 Sonuçlar

Sunan öğrencinin makaleden hesapladığı değerlere göre önerilen basit yöntemin sıkıştırma oranı yaklaşık 0,36'dır; gzip yaklaşık 0,35, bzip2 yaklaşık 0,26'dır. Yalnız Huffman ve dil modeli kullanılarak, ayrıntılı optimizasyon ya da uzantılar olmadan, dönemin yaygın sıkıştırıcılarına yaklaşılmıştır.

Eğitim cümlesi sayısı arttıkça sıkıştırılmış dosya boyutu düşer (ekrandan, 01:00:00): yaklaşık 400.000'den başlayan eğri hızla yaklaşık 200.000 civarına iner ve sonra düzleşir. Slayttaki açıklama, daha önce karşılaşılan trigramları görme olasılığının artmasıdır.

Soru: Eğrinin düşüşünün bittiği nokta kaç cümleye karşılık geliyor, 10.000 mi? **Cevap:** Evet. Hocaya göre en çok kullanılan trigramların sayısı sınırlıdır; 10.000'den sonra karşılaşılan yeni trigramlar çok azalır, bu yüzden sıkıştırma daha fazla iyileşmez.

Makale ayrıca perplexity arttıkça (yani dil modeli kötüleştikçe) sıkıştırılmış dosyanın büyüdüğünü göstermiştir. Hoca bunu entropiyle bağladı: entropi küçükse perplexity de küçük olur, entropi büyürse perplexity de yükselir. Çalışmanın amacı, dil modelinin sıkıştırma için işe yaradığını göstermektir. Gelecek çalışma olarak fazla boşlukların ve özel karakterlerin dikkate alınması önerilmiştir.

5. Veri Sıkıştırma Literatür Taraması

5.1 Makalenin yapısı

Dördüncü sunum, 1990'lardan 2018'e kadar olan literatürü tarayan 2018 tarihli bir derleme (survey) makalesini anlattı. Makale altı bölümden oluşur: giriş, ilişkili çalışmalar, matematiksel formüller ve başarı ölçütleri, sınıflandırma, gelecek yönleri ve özet. Sunum en çok sınıflandırma bölümü üzerinde durdu.

Veri sıkıştırmanın temel hedefi aktarılan veri miktarını azaltmak ve depolama alanından tasarruf etmektir. Makale sıkıştırmayı dört eksende inceler: veri kalitesi, kodlama şeması, veri tipi ve uygulama.

5.2 Veri kalitesine göre

Kayıpsız sıkıştırmada girdi ile çıktının aynı olması beklenir; metin, tıbbi görüntü ve uydu görüntüsü gibi küçük ayrıntıların önemli olduğu yerlerde kullanılır. Kayıplı sıkıştırmada girdi ile çıktının birebir uyması gerekmez; telefonda saklanan bir fotoğraftaki renklerin biraz kırpılması büyük sorun olmaz. Video sıkıştırma ve hiperspektral görüntüleme de kayıplı sıkıştırma kullanabilir.

5.3 Kodlama şemasına göre

Derste görülen yöntemler kısaca tekrar edildi:

- **Huffman:** bir ağaç kurulur, kodlar kökten yapraklara doğru okunur.
- **Aritmetik kodlama:** olasılık tabanlıdır; sembollerin gelme sırasına göre aralık küçültülerek minimum ve maksimum hesaplanır ve sonunda kesirli bir sayı elde edilir.
- **LZW:** sözlük tabanlıdır; sözlüğün boyu sabit ya da değişken olabilir, sözlüğe referans verilerek sıkıştırılır.
- **RLE:** bir sembol kaç kez tekrar ediyorsa sembol ve tekrar sayısı yazılır.

Derste işlenmeyen iki yöntem de kısaca anlatıldı. **Burrows-Wheeler dönüşümünde** girdi dizgesinin dönmeleri alt alta yazılıp alfabetik sıralanır ve sıralı tablonun son sütunu alınır; anlatılan yöntemde sıralama sütun sütun tekrarlanarak tablo oluşturulmaktadır. **Wavelet dönüşümünde** görüntü yüksek ve alçak frekansları geçiren filtrelerden geçirilerek dört parçaya bölünür; işlem tekrarlanarak giderek küçültülmüş bir görüntü elde edilir.

> **Ek bilgi (derste anlatılmadı):** Burrows-Wheeler dönüşümü tek başına veriyi küçültmez; aynı bağlamdaki karakterleri yan yana toplayarak sonraki move-to-front, RLE ve entropi kodlaması adımlarının daha iyi çalışmasını sağlar. bzip2 bu zinciri kullanır. Çözücünün asıl dizgeyi geri kurabilmesi için son sütunla birlikte asıl dizgenin sıralı tablodaki satır numarası da saklanır.

5.4 Veri tipine göre

Dört temel veri tipi vardır: metin, görüntü, ses ve video (ekrandan, 01:10:00). Metin sıkıştırmada kayıpsızlık gerekir; tek bir noktalama işareti ya da harf kaybedilmemelidir. Slayttaki sınıflandırma şemasında görüntü sıkıştırma altında EZW tabanlı, DCT tabanlı, Huffman tabanlı zincir kodu, tahmine dayalı (predictive) algoritma, iyileştirilmiş RLC, Burrows-Wheeler tabanlı ve SVD kullanan yöntemler gibi başlıklar; ses sıkıştırma altında IEEE 1857.2 ve Huffman shift coding; video sıkıştırma altında H.261, H.263 ve MPEG-1, MPEG-2, MPEG-3, MPEG-4, MPEG-7 yer alıyordu.

5.5 Uygulamaya göre

- **Kablosuz sensör ağları:** enerji tasarrufu gerekir ve veri büyüdükçe aktarımda sorun çıkar; veri miktarı azaltılarak daha verimli aktarım sağlanır.
- **Tıbbi görüntüler:** ayrıntılar çok önemlidir ve görüntüler büyüktür; bu yüzden ayrı incelenmiştir.

- **Özel uygulamalar:** büyük veri tabanlarının sıkıştırılması, gerçek zamanlı uygulamalar, çok veri üreten kayan noktalı sayı hesaplamaları ve beyin-bilgisayar arayüzlerinde aktarılan veriyi azaltmak için Walsh-Hadamard dönüşümü.

5.6 Gelecek yönleri

Makale üç başlık önerir:

- **Sıkıştırma tekniklerinin özellikleri:** dosya formatları çoğalmaktadır; sıkıştırmanın dosya formatından bağımsız yapılabilmesi gerekir.
- **Ayrık tonlu görüntüler:** bugün daha çok sürekli tonlu (continuous-tone) görüntüler üzerinde çalışılmaktadır; ayrık tonlu (discrete-tone) görüntülere de geçilmesi gerekir.
- **Makine öğrenmesi ve sıkıştırma:** makine öğrenmesi algoritmaları optimum bir değere ulaşmaya çalışır; sıkıştırma da veriyi en iyi nasıl küçültebileceğimizi sorar. Bu kesişimden yeni algoritmalar çıkabilir.

Soru: Makale son zamanlarda daha fazla çalışılan bir alanı (örneğin video sıkıştırma) öne çıkarıyor mu? **Cevap:** Uygulama temelli bir vurgu yoktur; en çok dosya formatlarının çoğalması noktasına, bir de makine öğrenmesine vurgu yapılmıştır.

Soru: Makine öğrenmesi sıkıştırmada nasıl kullanılıyor? Örnek veridir; ön işlem yapılır, özellik çıkarılır, etiketli ya da etiketsiz bir modelle öğrenme süreci işler. Sıkıştırma bunun neresine oturuyor? **Cevap:** Sunan öğrenci bunu sinir ağlarından çok, noktalar kümesine doğru uydurma gibi regresyon tarafında düşündüğünü söyledi. Hoca, “data compression” ile “machine learning” anahtar kelimeleriyle ayrıca arama yapılmasını önerdi.

6. Mespotine RLE (MRLE) ve İyileştirilmiş MRLE (iMRLE)

6.1 Sorun

Beşinci sunum, Hintli akademisyenlerin bir çalışmasını anlattı. RLE’de bazen sıkıştırılmış veri orijinal veriden büyük olur. Mespotine RLE (MRLE) bu olasılığı ortadan kaldırır: MRLE ile sıkıştırılan veri hiçbir zaman orijinal boyutun üzerine çıkmaz. Makalenin sonunda MRLE’nin de bir adım daha iyileştirilmesi olan iMRLE anlatılır.

Kayıplı sıkıştırmada insan gözünün algılayamayacağı veriler atılır; kayıpsızda bu imkân yoktur, örneğin tıbbi görüntülerde hiçbir veri kaybına tahammül edilemez. Bu yüzden kayıpsız tekniklerde sıkıştırılmış verinin orijinalden büyük çıkması önemli bir sorundur.

RLE hatırlatması (ekrandan, 01:20:00):

```
Orijinal : {B,C,A,D,D,D,D,D,E,E,E,E}   13 byte
RLE      : {1,B,1,C,1,A,6,D,4,E}          10 byte
```

Bu örnekte RLE başarılıdır; ama tekrarı az olan veri setlerinde RLE çıktısı büyür. Başarı tamamen verideki tekrar sayılarının azlığına ya da çokluğuna bağlıdır.

6.2 Comp-Bit kavramı

MRLE, her karakter için “Comp-Bit” (sıkıştırılabilir bit) adlı bir değer belirler (ekrandan, 01:21:15). Karakterin her tekrar grubu bir sayaç değeri alır:

Gruptaki tekrar sayısı	Sayaç
1	−1
2	0
3	1
4	2
5	3
6	4
7	5

Bir karakterin bütün gruplarının sayaçları toplanır. Toplam 0'dan büyükse karakter sıkıştırılabilir (Comp-Bit 1), 0 ya da küçükse sıkıştırılamaz (Comp-Bit 0). RLE'nin dezavantajı gördüğü her karakteri sıkıştırmasıydı; MRLE tekrar sayısı az olan karakterleri sıkıştırmaz, yalnız tekrarı çok olanları sıkıştırır.

Örnek dizi `DDAAABCAACBDAAADBBBBB` dört karakterden (A, B, C, D) oluşur (ekrandan, 01:21:15):

- **A:** 1. grup 3 tekrar (+1), 2. grup 2 tekrar (0), 3. grup 3 tekrar (+1). Toplam $1 + 0 + 1 = 2 > 0$, Comp-Bit 1.
- **B:** 1. grup 1 tekrar (−1), 2. grup 1 tekrar (−1), 3. grup 5 tekrar (+3). Toplam $-1-1+3 = 1 > 0$, Comp-Bit 1.
- **C:** iki grup, her biri 1 tekrar. Toplam $-1 - 1 = -2 < 0$, Comp-Bit 0.
- **D:** 1. grup 2 tekrar (0), 2. grup 1 tekrar (−1), 3. grup 1 tekrar (−1). Toplam $-2 < 0$, Comp-Bit 0.

MRLE Comp-Bit listesi **1100**'dür (4 bit).

6.3 MRLE ile sıkıştırma ve açma

Comp-Bit listesindeki 1'ler sıkıştırılabilir, 0'lar sıkıştırılamaz karakterlerdir (ekrandan, 01:23:55). D'nin Comp-Bit'i 0 olduğu için "DD" olduğu gibi yazılır. A'nın Comp-Bit'i 1 olduğu için "AAA" "A3" diye yazılır. Aynı işlem bütün karakterlere uygulanınca:

```
Orijinal : DDAAABCAACBDAAADBBBBB   = 21 karakter = 21 x 8 = 168 bit
MRLE     : DDA3B1CA2CB1DA3DB5      = 18 karakter = 18 x 8 = 144 bit
           + 4 Comp-Bit                            = 148 bit
```

Açarken Comp-Bit listesine bakılır. D'nin değeri 0'dır, D olduğu gibi kullanılır. A'nın değeri 1'dir; A'dan sonra gelen değer tekrar sayısıdır, A o kadar kez (örneğin 3 kez) yazılır. Bu işlem bütün karakterler için yapılınca orijinal veriye ulaşılır.

6.4 MRLE'nin dezavantajı ve iMRLE

MRLE, RLE'ye göre her durumda daha avantajlıdır, ama kendi dezavantajı vardır: tekrar sayısı azsa ve karakter sayısı çoksa Comp-Bit listesine ayrılan yer büyür. Örneğin hiç tekrar içermeyen `ABCDE` dizisinde Comp-Bit listesi 5 tane 0'dan oluşur; 40 bitlik veriye 5 bit eklenir ve 45 bit olur.

iMRLE, bu Comp-Bit listesinin boyutunu azaltır. Slayttaki örnekte 256 bitlik bir Comp-Bit listesi vardır; yalnız ilk ve son bit 1, diğerleri 0'dır, yani yalnız iki karakter tekrar etmiştir (ekrandan, 01:26:45):

- Liste 8 bitlik tamsayılara çevrilir. $10000000_2 = 128$ olduğundan integerData $= \{128, 0, 0, \ldots, 0, 1\}$ (32 değer) olur.
- numbersNew'deki sıfır olmayan değerlerden indexVal $= [128, 1]$ oluşturulur.
- numbersNew'deki 0'dan büyük her değer 1 yapılarak 32 bitlik cmpBitList1 $= 1000 \ldots 0001$ elde edilir.
- Bu 32 bit 4 byte'lık cmpBitList2 $= [128, 0, 0, 1]$'e çevrilir.
- cmpBitList2 ile indexVal birleştirilir: cmpBitListNew $= [128, 0, 0, 1, 128, 1]$. İlk 4 byte başlık, kalanlar indekstir.

Sonuçta 32 byte'lık MRLE Comp-Bit listesi 6 byte'lık iMRLE Comp-Bit listesine dönüşür. Makalenin özü budur: Comp-Bit listesinin boyutunu düşürmek. iMRLE ile de her durumda orijinal boyutun altına inilir; kayıpsız olduğu için özellikle tıbbi görüntülerde işe yarar.

> **Ek bilgi (derste anlatılmadı):** iMRLE'nin kazancı Comp-Bit listesinin seyrek olmasına bağlıdır. Sıfırdan farklı byte sayısı k ise iMRLE listesi yaklaşık $32/8 + k = 4 + k$ byte tutar; bu yüzden çok sayıda karakter sıkıştırılabilir çıktığında, yani liste seyrek olmadığında kazanç azalır.

7. CNN ile JPEG Sıkıştırma Kusurlarının Bastırılması

7.1 Problem

Altıncı sunum, derin evrişimli sinir ağıyla (CNN) görüntü sıkıştırma kusurlarının bastırılması üzerine bir makaleyi anlattı. Sıkıştırma yöntemleri iki kategoriye ayrılır: kayıpsız (örneğin PNG) ve kayıplı (örneğin JPEG). Kayıplı yöntemler geri döndürülemez bir sıkıştırma işlevine sahiptir, ama çok daha yüksek sıkıştırma oranı sağlar; web üzerinden iletilen ve depolanan görüntülerde yaygın kullanılır.

Yüksek sıkıştırma oranının bedeli, kullanıcı deneyimini bozan görsel kusurlardır (artefakt). Özellikle yüksek sıkıştırma oranlarında sıkıştırılmış ve orijinal görüntü arasındaki farklar, kullanılan sıkıştırma şemasına özgü efektlerle görünür hale gelir. Bu kusurlar yalnız göze hoş gelmemekle kalmaz, bilgisayarlı görme algoritmalarını da olumsuz etkiler. JPEG kayıplı görüntü sıkıştırmada en yaygın standarttır; aile fotoğraflarından tarihi belgelerin görüntülerine kadar çok sayıda JPEG görüntü depolanmaktadır. Sunan öğrenci, yüksek oranla sıkıştırılmış bir JPEG'de hem renklerde hem metinde kayıp olduğunu gösteren bir karşılaştırma koydu.

7.2 Önerilen çözüm

CNN'ler görüntü sınıflandırma, nesne algılama ve segmentasyon gibi görevlerde insan performansını bile aşmış; süper çözünürlük ve optik akış gibi düşük seviyeli regresyon görevlerinde de en iyi (state of the art) sonuçlara ulaşmıştır. Makale, JPEG sıkıştırılmış görüntülerdeki kusurları gidermek için yeni bir CNN mimarisi, bu derin ağı eğitmek için diğer düşük seviyeli görme görevlerine de uyarlanabilen yeni bir eğitim stratejisi önerir ve LIVE1 veri seti üzerinde kapsamlı değerlendirme yapar.

Çalışma, problemi süper çözünürlük ve optik akış tahmini gibi bir regresyon problemi olarak ele alır. Optik akış için geliştirilen FlowNet ve anlamsal segmentasyon için geliştirilen FCN'den esinlenerek hiyerarşik atlama bağlantılarına (skip connection) ve çok ölçekli kayıp fonksiyonuna sahip bir ağ önerilmiştir.

> **Ek bilgi (derste anlatılmadı):** Sunulan makale, sayılan yazar adlarına göre büyük olasılıkla Cavigelli, Hager ve Benini'nin 2017 tarihli "CAS-CNN: A Deep Convolutional Neural Network for Image Compression Artifact Suppression" çalışmasıdır. Ağ görüntüyü sıkıştırmaz; JPEG ile sıkıştırılıp açılmış görüntüyü girdi olarak alıp kusurları azaltılmış bir görüntü üretir, yani bir son işlem (post-processing) adımıdır.

7.3 Ağ mimarisi

Ağdaki her blok iki konvolüsyon katmanından oluşur; kanal sayısı önce 1'den 128'e, sonra 256'ya çıkar. Çözünürlük 2×2 adımlı (stride) konvolüsyonlar ve 2×2 ortalama havuzlama (average pooling) ile düşürülür (downsampling). Sonuçta 12 katmanlı bir CNN elde edilir. Ancak bu yapı standart eğitim yöntemleriyle en iyi doğruluğa ulaşacak biçimde eğitilemez; bu yüzden iki değişiklik yapılmıştır:

- **Atlama bağlantıları:** ağdaki ortalama yol uzunluğunu kısaltmak için, yüksek çözünürlüklü ara sonuçlar ağın daha önceki katmanlarından gelen düşük seviyeli özelliklerle birleştirilir. Bunun iki faydası vardır: yüksek çözünürlüklü çıktının üretilmesine yardımcı olacak ek bilgi sağlanır ve ara katmanlar atlanarak erken katmanların eğitimi kolaylaşır.
- **Ara çıktılar:** erken katmanların ağırlıklarının yeterince eğitilememesi sorununa karşı, ağın içinde tek bir konvolüsyon katmanıyla düşük çözünürlüklü görüntüler üretilir. Bu çıktılar doğrudan sistemden alınmak yerine en yakın komşu enterpolasyonuyla büyütülerek (upsample) özellik haritasına eklenir.

7.4 Performans ölçütleri

Görüntüler ve diğer sinyaller arasındaki farkları değerlendirmede en yaygın ölçütler ortalama kare hatası (MSE) ve bununla yakından ilişkili tepe sinyal-gürültü oranıdır (PSNR). MSE, bozuk ve referans görüntü arasındaki yoğunluk farkının karesinin piksel bazında ortalamasıdır. PSNR, desibel cinsinden ifade edilen, olası en büyük sinyal değerine normalize edilmiş MSE'dir (ekrandan, 01:40:00):

$$\mathrm{PSNR}(X, \hat{X}) = 10 \log_{10} \left(\frac{1}{\mathrm{MSE}(X, \hat{X})} \right)$$

$$\mathrm{MSE}(X, \hat{X}) = \frac{1}{|P|} \sum_{p \in P} e(x_p, \hat{x}_p)^2$$

MSE aynı zamanda eğitimde kayıp olarak kullanılmıştır. Diğer makalelerle karşılaştırma için iki ölçüt daha kullanılmıştır: yerel pencereler üzerinde parlaklık, kontrast ve yapı benzerliğini değerlendiren **SSIM** (Structural Similarity Index) ve MSE'ye blok etkisi faktörü (blocking effect factor) ekleyen **PSNR-B**.

7.5 Eğitim ve veri seti

Kalite faktörü 20 ile sıkıştırılmış görüntüler için ağ, çok ölçekli MSE kaybıyla test MSE'si yaklaşık 200. epoch'ta doygunluğa ulaşana kadar eğitilmiş, ardından yalnız çıktı görüntüsündeki MSE kaybıyla ince ayar (fine-tune) yapılmış ve test MSE'sinin en küçük olduğu parametreler seçilmiştir.

Eğitimde yaklaşık 396.000 eğitim ve 20.000 doğrulama görüntüsü içeren ImageNet 2013 veri seti kullanılmış, her görüntüden 120×120 piksellik kesitler alınmıştır. Testler LIVE1 veri setinin 29 görüntüsü üzerinde yapılmıştır. Ana eğitim kalite faktörü 20 için yapılmış; eğitim süresini kısaltmak için diğer kalite faktörlerine ait ağlar bu ağdan başlatılarak eğitilmiştir.

7.6 Sonuçlar

JPEG kalite faktörleri 10, 20, 40, 60 ve 80 için LIVE1 üzerinde ortalama PSNR, PSNR-B ve SSIM hesaplanmış, önceki çalışmalarla karşılaştırılmıştır. Önerilen ağ ölçütlerin hemen hepsinde en yüksek değerlere ulaşmış, PSNR-B'yi önceki çalışmalara göre 0,45 dB iyileştirmiştir. Piksel başına bit sayısına göre çizilen grafiklerde de her kalite faktöründe önerilen yöntemin JPEG'in üzerinde olduğu görülmektedir.

Önemli bir bulgu, belli bir kalite faktörü için eğitilen ağın başka kalite faktörlerinde de iyi çalışmasıdır: kalite faktörü 20 için eğitilen ağ, kalite faktörü 40 ile sıkıştırılmış görüntülerde de PSNR ve PSNR-B'yi iyileştirir; ağı her kalite faktörü için yeniden eğitmeye gerek yoktur. Görsel karşılaştırmalarda da önerilen yöntemin en iyi sonucu verdiği gösterilmiştir.

Özetle 12 katmanlı, hiyerarşik atlama bağlantılı ve çok ölçekli kayıpla eğitilen bu CNN, PSNR'de sıradan JPEG'e göre 1,79 dB'e, önceki en iyi CNN sonucuna göre 0,36 dB'e kadar iyileşme sağlamıştır. Yöntem JPEG'e özel hazırlanmadığı için başka kayıplı sıkıştırma algoritmalarına da uygulanabilir.

Örnek Sorular

Soru 1 (ekrandan, 00:15:15): Aşağıdaki 8×8 bloğun en küçük değeri 188'dir. Önerilen neredeyse kayıpsız yöntemle (en küçüğü çıkar, tek değerlerden 1 çıkar, 2'ye böl) ilk satır `188 231 188 230 188 230 188 230` hangi değerlere dönüşür?

> **Çözüm:** En küçüğü çıkarınca `0 43 0 42 0 42 0 42`; tek değerden 1 çıkarınca `0 42 0 42 0 42 0 42`; 2'ye bölünce `0 21 0 21 0 21 0 21`. Bloğun tamamında yalnız 0, 21 ve 3 değerleri kalır; Huffman bunlara en sık olandan başlayarak kısa kodlar verir.

Soru 2 (ekrandan, 00:28:50): `8 8 8 8 8 96 12 45 45 45 45` dizisini kaçış kodu 255 olan RLE ile kodlayın.

> **Çözüm:** `255 5 8 96 12 255 4 45`. Tekrar eden diziler "kaçış, adet, değer" üçlüsüyle, tek değerler olduğu gibi yazılır; 11 değer 8 değere iner.

Soru 3 (ekrandan, 00:29:50): Olasılıklar a = 0,2, b = 0,3, c = 0,1, d = 0,4 ise `b d a b` dizisinin aritmetik kodlamadaki son aralığı nedir?

Çözüm: [0,2; 0,5) → [0,38; 0,5) → [0,38; 0,404) → [0,3848; 0,392). Bu aralıktaki herhangi bir sayı diziyi temsil eder.

Soru 4 (ekrandan, 00:32:45 ve 00:33:40): 13 sayısının Elias gamma kodunu ve patlak Elias (P1) kodunu bulun.

Çözüm: $13 = 1101_2$ (4 bit). Gamma: önüne 3 tane 0, kod **0001101**. P1: bitler ters çevrilir, 1011; üç tane 1 için 111 ve sonuna 0, önek 1110; kod **11101011**.

Soru 5 (ekrandan, 00:36:30): 3 sayısının Goldbach G0 kodunu bulun.

Çözüm: $2(3+3) = 12 = 5+7$. Asal sırası [3][5][7] için bitler 0, 1, 1; kod **011**.

Soru 6 (ekrandan, 00:39:10): 8 sayısının Goldbach G1 kodunu bulun (asal tablosu 1, 3, 5, 7, 11, ...).

Çözüm: $8 = 3+5$; indeksler $(i,j) = (2,3)$; çift $(i, j-i+1) = (2,2)$; $\gamma(2) = 010$; kod **010 010**.

Soru 7 (derste sözlü, 00:43:14): Bir yöntemle 40 bitlik veri 80 bite çıkmışsa sıkıştırma oranı nedir ve bu bir sıkıştırma mıdır?

Çözüm: Oran çıktı/girdi $= 80/40 = \%200$'dür. %100'ün üzerindeki oran büyüme demektir; sıkıştırma yoktur.

Soru 8 (ekrandan, 01:21:15 ve 01:23:55): `DDAAABCAACBDAAADBBBBB` için MRLE Comp-Bit listesini ve sıkıştırılmış veriyi bulun. Kaç bit kazanılır?

Çözüm: Sayaçlar (1 tekrar −1, 2 tekrar 0, 3 tekrar +1, ...) toplanır: A $= 1+0+1 = 2$, B $= -1-1+3 = 1$, C $= -2$, D $= 0-1-1 = -2$. Comp-Bit listesi **1100**. Sıkıştırılmış veri `DDA3B1CA2CB1DA3DB5` (18 karakter, 144 bit) + 4 Comp-Bit = 148 bit. Orijinal 21 × 8 = 168 bit; kazanç 20 bittir.

Soru 9 (ekrandan, 01:26:45): Yalnız ilk ve son biti 1 olan 256 bitlik bir Comp-Bit listesi iMRLE ile kaç byte'a iner?

Çözüm: 8 bitlik tamsayılar: [128, 0, ..., 0, 1] (32 değer). indexVal = [128, 1]; cmpBitList2 = [128, 0, 0, 1]; birleşim [128, 0, 0, 1, 128, 1]. 32 byte yerine **6 byte**.

Soru 10 (derste sözlü, 00:16:19): Neredeyse kayıpsız blok yönteminde tek değerlerden 1 çıkarmanın amacı nedir?

Çözüm: Sunumdaki cevaba göre değerlerin bir sonraki adımda 2'ye tam bölünebilmesidir. Bu adım yöntemi tam kayıpsız olmaktan çıkarır: geri kurulan değer orijinal tek değerden 1 eksik olabilir.

Dersin Toparlaması

Bu derste altı seminer sunumuyla veri sıkıştırmanın farklı yönleri ele alındı. Görüntü tarafında, bloklara bölüp en küçük değeri çıkararak ve değerleri yarıya indirerek Huffman'a daha küçük bir alfabe veren neredeyse kayıpsız bir yöntem ile JPEG'in yüksek sıkıştırma oranlarında oluşturduğu kusurları gideren 12 katmanlı bir CNN görüldü. Metin tarafında RLE, aritmetik kodlama ve iki yeni önek kod (patlak Elias kodu ve Goldbach kodları) karşılaştırıldı; dil modeline dayalı Huffman ağaçlarıyla kelime düzeyinde sıkıştırma ve perplexity ile entropi arasındaki bağ (2^H) anlatıldı. RLE'nin veriyi büyütme sorununu Comp-Bit listesiyle çözen MRLE ve bu listenin kendisini küçülten iMRLE ayrıntılı örneklerle verildi. Literatür taraması sunumu ise sıkıştırma yöntemlerini veri kalitesi, kodlama şeması, veri tipi ve uygulama eksenlerinde sınıflandırarak makine öğrenmesiyle sıkıştırmanın kesişimini gelecek yönü olarak gösterdi.

Hocanın sunumlar boyunca altını çizdiği ortak nokta, sıkıştırma oranının doğru yorumlanması (çıktı/girdi; %100'ün üstü büyümedir) ve bir yöntemin her adımının gerekçesinin sorgulanmasıdır.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.